Consistency Proofs: Wire Formats and Backward Compatibility Policy

Overview

Transparency logs prove that the log is append-only through **Merkle consistency proofs**. When a client pins a checkpoint at tree size m with root R_m and later receives a checkpoint at tree size n ($n \geq m$) with root R_n, a consistency proof demonstrates that the first m leaves of the new tree match the original m leaves, without requiring the client to download the full log.

SatsPath supports two versions of the consistency proof wire format:

Property	Version 1 (V1 - Legacy/Bounded)	Version 2 (V2 - Compact RFC 6962)
Proof Payload	Entire sequence of leaf hashes (n hashes)	Minimal subtree hashes ($O(\log n)$ hashes)
Bandwidth	$O(n)$	$O(\log n)$
Verification Memory	$O(n)$	$O(\log n)$
Max Tree Size Cap	16,384 leaves	Unlimited (scales with log depth)
Algorithm	Full Merkle tree recomputation	RFC 6962 Section 2.1.2 Subproof verification
Server Trust	Zero (Pure verifier)	Zero (Pure verifier)

Wire Format (`MerkleConsistencyProof`)

```
{
  "version": 2,
  "old_tree_size": 1000,
  "new_tree_size": 2500,
  "old_root": "a1b2...",
  "new_root": "c3d4...",
  "audit_path": [
    "e5f6...",
    "7a8b..."
  ]
}
```

Fields

- `version` (`u16`): Proof format version (`1` or `2`).
- `old_tree_size` (`u64`): Tree size m at the prior pinned checkpoint ($m > 0$).
- `new_tree_size` (`u64`): Tree size n at the new checkpoint ($n \geq m$).
- `old_root` (`String`, hex): Expected root hash at size m.
- `new_root` (`String`, hex): Expected root hash at size n.
- `audit_path` (`Vec<String>`, hex):
  - In **V1**: Contains all n leaf hashes of the new tree.
  - In **V2**: Contains the $O(\log n)$ intermediate sibling/subtree node hashes specified by RFC 6962.

Backward Compatibility Policy

1. Proof Generation (`TransparencyLog::consistency`)

- All new consistency proofs produced by `TransparencyLog::consistency(old_size, new_size)` are emitted using **Version 2 (RFC 6962 compact)**.
- Tree size is bounded only by the actual log size; no arbitrary 16,384 leaf ceiling is applied to V2 generation.

2. Verifier Support (`verify_consistency_proof`)

- **Version 2 proofs (RFC 6962)**:
  - Verified according to RFC 6962 consistency proof arithmetic.
  - Verification operates in $O(\log n)$ time and $O(1)$ auxiliary memory per step.
  - Pure verification: Does not trust the server or log operator.
  - Bit-flips, truncated paths, reordered paths, or tampered roots fail closed.
- **Version 1 proofs (Bounded Legacy)**:
  - Verifier checks that `new_tree_size <= 16,384`.
  - Verifier validates that `audit_path.len() == new_tree_size`.
  - Recomputes `merkle_root(&audit_path[..old_tree_size]) == old_root` and `merkle_root(&audit_path) == new_root`.
  - Any V1 proof with `new_tree_size > 16,384` is **rejected immediately** prior to slicing or allocating memory, preserving DDoS resistance against malformed V1 payloads.

3. Checkpoint Binding and Local Pinning

- Consistency proof verification remains strictly bound to signed checkpoints and local pinned state.
- Transition checks (`verify_checkpoint_transition`) ensure:
  - `old_tree_size` matches the locally pinned checkpoint tree size.
  - `new_tree_size` matches the candidate checkpoint tree size.
  - `old_root` matches the locally pinned checkpoint root.
  - `new_root` matches the candidate checkpoint root.
  - Operator key continuity and sequence numbers are preserved.